DataOps Take-Home: Podcast Speech Dataset Pipeline

Summary

Build a data-processing pipeline that converts public podcast audio into a LibriLight-style speech dataset: short, clean, single-speaker clips with useful metadata.

You have a **24-hour elapsed window**. We expect roughly **6-8 hours of active work**, not an overnight grind. We will reimburse up to **$100 in actual cloud or processing costs**.

Your main optimization metric is:

> **the amount of usable single-speaker processed audio produced within the 24-hour window and $100 reimbursement budget**

Your submission will be used to evaluate this take-home, not as production training data unless separately agreed. Do not include secrets, API keys, or private credentials. If reimbursement logistics are a problem, tell us before starting.

Task

Use PodcastIndex, or another PodcastIndex-backed source, to find and process podcast audio into utterance-level clips.

The target output is a dataset with many clean speech clips:

- mostly under 10 seconds
- with a long tail up to about 30 seconds
- generally no clips longer than 30 seconds
- one speaker at a time
- no silence-only, music-only, ad-only, intro/outro, or overlapping-speaker segments where your pipeline can detect them

You may use AWS, GCP, Azure, RunPod, Lambda, spot instances, local processing, or any other stack you think is best.

Output Format

Submit audio clips plus a metadata manifest. JSONL is preferred.

Each row should look roughly like:

```
{
  "clip_id": "podcast123_episode456_000012",
  "podcast_id": "podcast123",
  "podcast_title": "Example Podcast",
  "episode_id": "episode456",
```

```
    "episode_title": "Example Episode",
    "source_url": "https://...",
    "start_seconds": 123.45,
    "end_seconds": 131.82,
    "duration_seconds": 8.37,
    "speaker_id": "podcast123_speaker_002",
    "utterance": "transcript text if generated, otherwise null",
    "audio_path": "s3://bucket/path/clip.flac",
    "quality_flags": {
      "vad_confidence": 0.94,
      "overlap_detected": false,
      "music_detected": false,
      "discard_reason": null
    }
  }
```

Also include a processing summary with raw hours ingested, usable speech hours produced, clip count, duration distribution, discard reasons, and total cost.

Requirements

Your pipeline should:

1. **Ingest podcast audio**
   - Use PodcastIndex or another PodcastIndex-backed source.
   - Track podcast, episode, source URL, and download/streaming metadata.
   - Report how many raw podcast hours were ingested.
2. **Detect speech and remove non-speech**
   - Remove silence and long pauses.
   - Remove or discard music-only sections, ads, intros/outros, noisy sections, and other non-clean speech where possible.
   - Explain which filters are automated and which are heuristic.
3. **Split audio into clean utterance clips**
   - Clips should contain one speaker at a time.
   - Overlapping speech, crosstalk, and simultaneous speakers should be removed or discarded where possible.
   - Most clips should be under 10 seconds.
   - There should be a meaningful long tail of clips between 10 and 30 seconds.
   - Longer clips may go up to around 30 seconds.
   - Clips should generally not exceed 30 seconds.
4. **Assign speaker IDs**
   - Assign speaker IDs where possible.
   - Speaker IDs should be persistent across episodes of the same podcast where possible.
   - At minimum, make a real attempt at cross-episode speaker matching using diarization, speaker embeddings, clustering, similarity matching, or another defensible method.
   - This does not need to be perfect, but the approach and known failure modes should be explicit.
5. **Generate metadata**

- Include podcast and episode source.
- Include source URL.
- Include clip start/end timestamps.
- Include duration.
- Include speaker ID if available.
- Include transcript if generated.
- Include output audio path.
- Include quality/filtering flags where useful.

6. **Store the result**
   - Store processed clips and metadata in a cloud bucket, object store, shared drive, or other accessible location.
   - The result should be easy for us to inspect and download.
7. **Make the pipeline reproducible**
   - Submit code, scripts, notebooks, Dockerfiles, configs, or commands needed to rerun the pipeline.
   - Include enough setup notes that another engineer could rerun your approach.
8. **Show DataOps judgment**
   - Make processing resumable or checkpointed where practical.
   - Avoid duplicated work when rerunning failed jobs.
   - Track metrics for throughput, yield, cost, and failure rates.
   - Call out the main bottleneck: download bandwidth, decode time, model inference, storage writes, GPU availability, queueing, or something else.
   - Explain what you would monitor in production.

Cost Analysis

Include both:

Trial-run cost report

Report:

- cloud provider or local hardware used
- instance types, services, storage, and regions
- wall-clock runtime
- raw podcast hours ingested
- usable speech hours produced
- number of clips produced
- total cost incurred
- cost per raw podcast hour
- cost per usable processed speech hour

- storage costs
- network or egress costs
- failed runs or wasted spend, if any

Please include receipts, billing screenshots, usage exports, or enough detail for reimbursement verification. We reimburse up to **$100**.

At-scale estimate

Estimate:

- cost per raw podcast hour ingested
- cost per usable processed speech hour
- compute costs
- storage costs
- network and egress costs
- expected bottlenecks
- expected throughput at larger scale
- what you would change for 10,000+ raw podcast hours
- what metrics, alerts, retries, and data-quality checks you would add in production

Deliverables

Submit within 24 hours:

1. Short writeup explaining your approach and tradeoffs.
2. Link to processed dataset output.
3. Processing code or repo.
4. Sample metadata manifest.
5. Processing summary: raw hours, usable hours, clip count, yield, duration distribution, discard reasons.
6. Trial-run cost breakdown.
7. At-scale cost estimate.
8. Speaker ID / speaker matching explanation.
9. Notes on resumability, failure handling, monitoring, and production bottlenecks.
10. Known limitations, especially around diarization, transcript quality, speaker consistency, overlapping speech, ads/music/noise, and filtering accuracy.

Evaluation

We will evaluate:

- **Throughput and cost discipline:** amount of usable single-speaker speech produced within the 24-hour window and $100 reimbursement budget.
- **Data quality:** clean, speech-heavy, single-speaker, correctly segmented clips.
- **Scalability:** whether the architecture could plausibly process much larger volumes.

- **Operational maturity:** checkpoints, metrics, failure handling, and productionization thinking.
- **Cost reasoning:** specific and credible actual and at-scale cost analysis.
- **Reproducibility:** whether we can understand and rerun the pipeline.
- **Pragmatism:** sensible tradeoffs under a short time constraint.

Use off-the-shelf tools where they help. VAD, diarization, ASR, speaker embeddings, clustering, ffmpeg, batch jobs, queues, and object storage are all fair game. Transcript quality is useful, but the core task is high-throughput, cost-aware production of clean single-speaker speech clips.